

COSMOS

PRACTICAL SOLUTION OF JAVASCRIPT

AN ISO 9001:2015 CERTIFIED INSTITUTE

COSMOS

The Real Training Centre

COSMOS

4th Floor, Shiva Blessing – 3, Valentine Circle, Waghawadi Road, Bhavnagar. 8154910092

➤ Introduction to JavaScript

1. **Print "Hello, World!" to the console.**

```
console.log("Hello, World!");
```

2. **Print Your Bio Data on console.**

```
console.log("Your Data");
```

➤ Variables and Data Types

3. **Create variables for a name, age, and a boolean for whether you are a student or not.**

```
let name = "John";  
let age = 25;  
let isStudent = true;  
console.log(name); // string  
console.log(age); // number  
console.log(isStudent); // boolean
```

4. **Use typeof to check the data types.**

```
let name = "John";  
let age = 25;  
let isStudent = true;  
console.log(typeof(name)); // string  
console.log(typeof(age)); // number  
console.log(typeof(isStudent)); // boolean
```

➤ Basic Operations

5. **Perform addition, subtraction, multiplication, and division with two numbers.**

```
let num1 = 10, num2 = 5;  
console.log(num1 + num2); // 15  
console.log(num1 - num2); // 5  
console.log(num1 * num2); // 50  
console.log(num1 / num2); // 2
```

6. **Concatenate two strings and log them.**

```
let greeting = "Hello";  
let name = "John";  
console.log(greeting + " " + name); // Hello John
```

➤ Conditionals: if, else, switch

7. Create a for loop that prints numbers 1-10.

```
// 1. for loop to print 1-10
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

8. Create a while loop to print numbers 1-5.

```
// 2. while loop to print 1-5
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

9. Use forEach() to log each element of an array.

```
// 3. forEach() to log each element
let arr = [1, 2, 3, 4, 5];
arr.forEach((item) => {
  console.log(item);
});
```

➤ Arrays: Introduction

10. Create an array of your favorite foods and log each item using a for loop.

```
let foods = ["Pizza", "Burger", "Sushi"];
for (let i = 0; i < foods.length; i++) {
  console.log(foods[i]);
}
```

11. Add and remove elements from the array using push() and pop().

```
foods.push("Pasta");
console.log(foods); // ["Pizza", "Burger", "Sushi", "Pasta"]
foods.pop();
console.log(foods); // ["Pizza", "Burger", "Sushi"]
```

➤ Array Methods: Advanced

12. Use `map()` to double each element in an array.

```
// 1. map() to double each element
let doubled = numbers.map((num) => num * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

13. Use `filter()` to return only even numbers from an array.

```
// 2. filter() to return only even numbers
let evenNumbers = numbers.filter((num) => num % 2 === 0);
console.log(evenNumbers); // [2, 4]
```

14. Use `reduce()` to sum up the elements in an array.

```
// 3. reduce() to sum up the elements
let sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum); // 15
```

➤ Objects: Introduction

15. Create an object representing a book with properties like title, author, and year.

```
// 1. Create a book object
let book = {
  title: "JavaScript: The Good Parts",
  author: "Douglas Crockford",
  year: 2008
};
```

16. Modify one of the properties and log the updated object.

```
// 2. Modify a property
book.year = 2020;
console.log(book);
```

➤ Objects: Advanced

17. Use object destructuring to extract the title and author from a book object.

```
// 1. Object destructuring
let book = {
  title: "JavaScript: The Good Parts",
  author: "Douglas Crockford",
  year: 2008
};
```

```
let { title, author } = book;
console.log(title); // JavaScript: The Good Parts
console.log(author); // Douglas Crockford
```

18. Create an object with a method that logs the details of a book.

```
// 2. Object with method
let bookWithMethod = {
  title: "JavaScript: The Good Parts",
  author: "Douglas Crockford",
  logDetails: function() {
    console.log(this.title + " by " + this.author);
  }
};

bookWithMethod.logDetails(); // JavaScript: The Good Parts by Douglas Crockford
```

➤ Functions:

19. Write an anonymous function to add two numbers.

```
// 1. Anonymous function
let add = function(num1, num2) {
  return num1 + num2;
};
console.log(add(2, 3)); // 5
```

20. Write the same function using an arrow function.

```
// 2. Arrow function
let addArrow = (num1, num2) => num1 + num2;
console.log(addArrow(2, 3)); // 5
```

➤ Scope and Closures:

21. Create a function that has a local variable, then define another function inside it that accesses the local variable.

```
function outer() {
  let outerVariable = "I'm outside!";
  function inner() {
    console.log(outerVariable); // Accessing outer function's variable
  }
  inner();
}
outer(); // I'm outside!
```

➤ Order Functions:

22. Write a higher-order function that takes another function as an argument and applies it to an array of numbers.

```
function applyFunctionToArray(arr, func) {  
  return arr.map(func);  
}  
  
let numbers = [1, 2, 3, 4];  
let result = applyFunctionToArray(numbers, (num) => num * 2);  
console.log(result); // [2, 4, 6, 8]
```

➤ Callbacks:

23. Create a function greet that takes a callback to say a personalized greeting.

```
function greet(name, callback) {  
  callback(name);  
}  
greet("John", function(name) {  
  console.log("Hello, " + name + "!");  
}); // Hello, John!
```

➤ Promises:

24. Create a promise that resolves after 2 seconds and logs a success message.

```
let promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Success!"), 2000);  
});
```

25. Handle errors using .catch().

```
promise  
  .then((message) => console.log(message)) // Success!  
  .catch((error) => console.log(error));
```

➤ Async/Await:

26. Write an async function that fetches data (simulated with a delay using setTimeout).

```
async function fetchData() {  
  let data = await new Promise((resolve) => {  
    setTimeout(() => resolve("Fetched Data"), 2000);  
  });  
  console.log(data); // Fetched Data
```

```
}
```

```
fetchData();
```

27. Use await to handle asynchronous execution.

```
async function fetchData() {  
  let data = await new Promise((resolve) => {  
    setTimeout(() => resolve("Fetched Data"), 2000);  
  });  
  console.log(data); // Fetched Data  
}
```

```
fetchData();
```

➤ Let & Const:

28. Compare let and const by declaring variables in different scopes and modifying them.

```
function testScope() {  
  let a = 5;  
  const b = 10;  
  
  console.log(a); // 5  
  console.log(b); // 10  
  
  a = 15; // `let` can be reassigned  
  console.log(a); // 15  
  
  // b = 20; // This will cause an error because `const` cannot be reassigned  
}  
testScope();  
  
// Example of `let` in a block scope  
if (true) {  
  let x = 10;  
  console.log(x); // 10  
}  
// console.log(x); // This will cause an error because `x` is not defined outside the block
```

➤ Destructuring:

29. Use array destructuring to get the first and second elements of an array.

```
// 1. Array Destructuring  
let arr = [10, 20, 30];  
let [first, second] = arr;  
console.log(first); // 10
```

```
console.log(second); // 20
```

30. Use object destructuring to extract properties from an object.

```
// 2. Object Destructuring
let person = { name: "John", age: 30, job: "Developer" };
let { name, age } = person;
console.log(name); // John
console.log(age); // 30
```

➤ Template Literals:

31. Create a multi-line string using template literals.

```
// 1. Multi-line string using template literals
let multiLineString = `This is a
multi-line string
using template literals.`;
console.log(multiLineString);
```

32. Use string interpolation to include variables in a string.

```
// 2. String interpolation
let name = "Alice";
let age = 25;
let greeting = `Hello, my name is ${name} and I am ${age} years old.`;
console.log(greeting); // Hello, my name is Alice and I am 25 years old.
```

➤ Regular Expressions:

33. Write a regex pattern to check if an email is valid

```
// 1. Regex to check if an email is valid
let email = "test@example.com";
let emailRegex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
console.log(emailRegex.test(email)); // true
```

34. Use regex to extract digits from a string.

```
// 2. Regex to extract digits from a string
let str = "The year is 2024 and it's a leap year.";
let digitRegex = /\d+/g;
let digits = str.match(digitRegex);
console.log(digits); // ["2024"]
```

➤ Error Handling:

35. Write a function that catches errors when trying to parse invalid JSON.

```
// 1. Catching errors while parsing JSON
function parseJSON(jsonString) {
  try {
    let data = JSON.parse(jsonString);
    console.log("Parsed Data:", data);
  } catch (error) {
    console.log("Error parsing JSON:", error.message);
  }
}
parseJSON('{ "name": "John" }'); // Parsed Data: { name: 'John' }
parseJSON('{ "name": "John"'); // Error parsing JSON: Unexpected token } in JSON at position 15
```

36. Throw a custom error when a function receives an invalid argument.

```
// 2. Throwing a custom error
function checkAge(age) {
  if (age < 18) {
    throw new Error("Age must be at least 18.");
  }
  console.log("Age is valid.");
}

try {
  checkAge(15); // Throws an error
} catch (error) {
  console.log(error.message); // Age must be at least 18.
}
```

➤ The DOM: Introduction:

37. Create a simple HTML page and select an element using JavaScript to change its content.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Exercise</title>
</head>
<body>
  <h1 id="heading">Original Heading</h1>
  <script src="script.js"></script>
</body>
```

```
</html>
// JavaScript: script.js
let heading = document.getElementById("heading");
heading.textContent = "Updated Heading using JavaScript!";
```

➤ **DOM Events:**

38. Add a button to your page and create an event listener to change the text when clicked.

```
<!-- HTML: index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Event</title>
</head>
<body>
  <button id="myButton">Click Me!</button>
  <h1 id="message">Hello!</h1>

  <script src="script.js"></script>
</body>
</html>
// JavaScript: script.js
let button = document.getElementById("myButton");
let message = document.getElementById("message");
button.addEventListener("click", function() {
  message.textContent = "Button Clicked!";
});
```

➤ **DOM Manipulation: Advanced:**

39. Create a dynamic list of items and append them to a webpage.

```
<!-- HTML: index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation</title>
</head>
<body>
  <ul id="itemList"></ul>
  <script src="script.js"></script>
</body>
</html>
```

```
// JavaScript: script.js
let itemList = document.getElementById("itemList");

let items = ["Item 1", "Item 2", "Item 3"];

items.forEach(item => {
  let listItem = document.createElement("li");
  listItem.textContent = item;
  listItem.addEventListener("click", function() {
    listItem.remove();
  });
  itemList.appendChild(listItem);
});
```

40. Remove an item from the list when clicked.

```
<!-- HTML: index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation</title>
</head>
<body>
  <ul id="itemList"></ul>
  <script src="script.js"></script>
</body>
</html>

// JavaScript: script.js
let itemList = document.getElementById("itemList");

let items = ["Item 1", "Item 2", "Item 3"];

items.forEach(item => {
  let listItem = document.createElement("li");
  listItem.textContent = item;
  listItem.addEventListener("click", function() {
    listItem.remove();
  });
  itemList.appendChild(listItem);
});
```

➤ Working with JSON:

41. Convert a JavaScript object to a JSON string.

```
// 1. Convert JavaScript object to JSON string
let person = { name: "Alice", age: 25 };
let jsonString = JSON.stringify(person);
console.log(jsonString); // {"name":"Alice","age":25}
```

42. Parse a JSON string into a JavaScript object.

```
// 2. Parse JSON string into JavaScript object
let jsonString = '{"name":"Alice","age":25}';
let parsedObject = JSON.parse(jsonString);
console.log(parsedObject); // { name: 'Alice', age: 25 }
```

➤ Classes and Inheritance:

43. Create a Person class with properties and a method, then create a Student subclass that inherits from Person.

```
// 1. Person class
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
  }
}

// 2. Student subclass
class Student extends Person {
  constructor(name, age, school) {
    super(name, age); // Call the parent class constructor
    this.school = school;
  }

  introduce() {
    console.log(`${this.name} studies at ${this.school}.`);
  }
}

let student = new Student("Alice", 20, "XYZ University");
```

```
student.greet(); // Hello, my name is Alice and I am 20 years old.  
student.introduce(); // Alice studies at XYZ University.
```

➤ Modules and Imports:

44. Split your code into two files: one with a function and another that imports and calls the function.

```
// File: math.js (Module)  
export function add(a, b) {  
    return a + b;  
}  
// File: app.js (Main File)  
import { add } from './math.js';  
console.log(add(5, 3)); // 8
```

➤ Local Storage:

45. Store a value in localStorage and retrieve it later.

```
// Store data in localStorage  
localStorage.setItem("name", "John");  
  
// Retrieve data from localStorage  
let name = localStorage.getItem("name");  
console.log(name); // John
```

➤ Event Delegation:

46. Create a list of items and use event delegation to log which item is clicked.

```
<!-- HTML -->  
<ul id="itemList">  
    <li>Item 1</li>  
    <li>Item 2</li>  
    <li>Item 3</li>  
</ul>  
<script src="script.js"></script>  
javascript  
Copy code  
// JavaScript: script.js  
let itemList = document.getElementById("itemList");  
  
itemList.addEventListener("click", function(event) {  
    if (event.target.tagName === "LI") {  
        console.log("Item clicked: " + event.target.textContent);  
    } });
```

➤ Final Project

47. Build a To-Do list where you can add and delete tasks. Use localStorage to persist tasks

```
<!-- HTML -->
<input type="text" id="taskInput" placeholder="Add a task">
<button id="addTask">Add Task</button>
<ul id="taskList"></ul>
<script src="script.js"></script>
javascript
Copy code
// JavaScript: script.js
let addTaskButton = document.getElementById("addTask");
let taskInput = document.getElementById("taskInput");
let taskList = document.getElementById("taskList");

// Load tasks from localStorage
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
tasks.forEach(task => {
  addTaskToList(task);
});

// Add new task
addTaskButton.addEventListener("click", function() {
  let taskText = taskInput.value;
  if (taskText) {
    tasks.push(taskText);
    localStorage.setItem("tasks", JSON.stringify(tasks));
    addTaskToList(taskText);
    taskInput.value = "";
  }
});

// Add task to the DOM
function addTaskToList(task) {
  let li = document.createElement("li");
  li.textContent = task;
  li.addEventListener("click", function() {
    li.remove();
    tasks = tasks.filter(t => t !== task);
    localStorage.setItem("tasks", JSON.stringify(tasks));
  });
  taskList.appendChild(li);
}
```